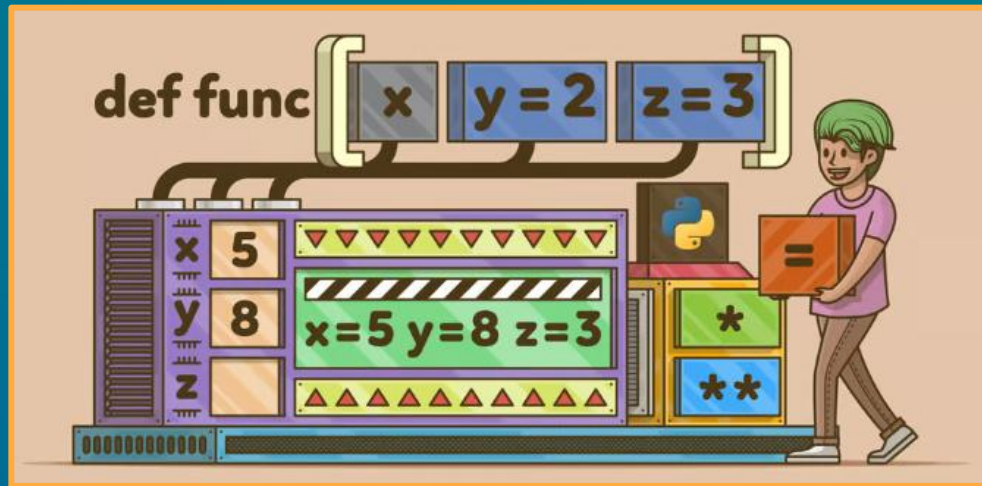


Funciones



Introducción

En Python, una función es un bloque de código que realiza una tarea específica.

¿Qué es una función?

A medida que los programas crecen en extensión y complejidad la resolución se torna más **complicada** y su **depuración** y **modificaciones** futuras resultan casi imposibles. Para resolver este tipo de problemas lo que se hace es **dividir el programa** en módulos más pequeños que cumplan **una tarea simple y bien acotada** llamados funciones.

¿Para qué sirve una función?

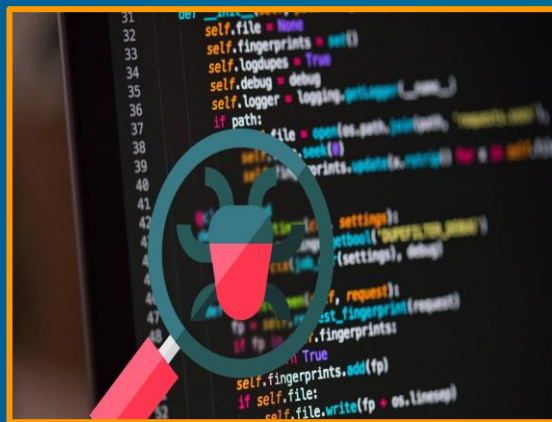
- Las funciones sirven para **descomponer una tarea específica en un bloque de código** que se puede llamar varias veces desde cualquier parte del programa.
- Las funciones permiten que el código sea **modular, legible y fácil de mantener**.

Objetivos de una función:



Minificación: Logramos que el programa sea más simple de comprender ya que cada función se dedica a realizar una tarea en particular.

Objetivos de una función:



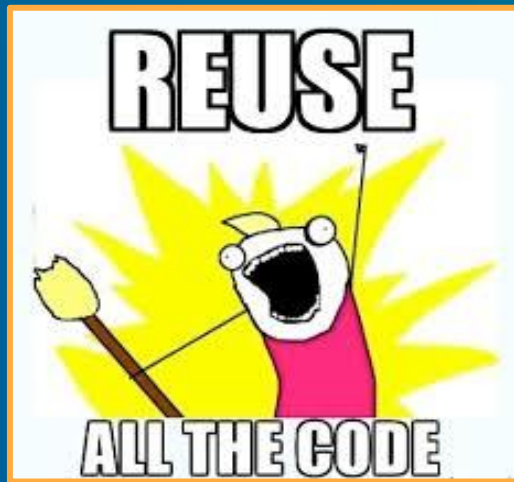
Depuración: La depuración queda acotada a cada función.

Objetivos de una función:



Modificación: Las modificaciones al programa se reducen a cada módulo.

Objetivos de una función:



Reutilización: Cuando cada función está bien probada, se la puede usar las veces que se quiera y así reutilizar código.

Objetivos de una función:



Independencia: Se obtiene una autonomía del código donde cada función es independiente de otra.

Componentes de una función:

Palabra reservada

Nombre

Parámetro

Parámetro opcional

```
def calcular_precio_con_iva(valor_sin_iva, iva=21):
```

```
    '''
```

```
    Documentacion
```

```
    '''
```

```
    resultado = valor_sin_iva * (1 + (iva / 100))
```

```
    return resultado
```

Documentación

Variable local

Valor del retorno

Desarrollo e invocación

```
def calcular_precio_con_iva(valor_sin_iva, iva = 21):  
    resultado = valor_sin_iva * (1 + iva / 100)  
    return resultado
```

Definición y
desarrollo

```
print(calcular_precio_con_iva(100))    # 121.0  
print(calcular_precio_con_iva(100, 10.5)) # 110.5
```

Invocación o
llamada

Palabras reservadas: **def** y **return**

El uso de **def** nos permite crear una función.
Podemos utilizar **return** para devolver un valor.

```
def sumar(numero_a, numero_b):  
    suma = numero_a + numero_b  
    return suma
```

```
resultado = sumar(33, 2)  
print("El resultado de la suma es", resultado)
```

Nombre

Las funciones y las variables se deberán escribir en formato **snake_case**.

Las palabras separadas entre sí por barra baja (underscore) en vez de espacios y las letras en minúscula.

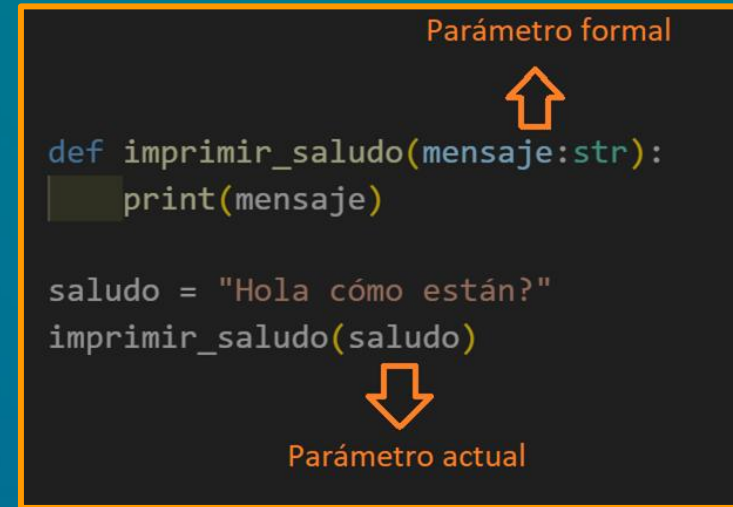
Las funciones representan una acción, por lo tanto, el nombre debe contener un verbo.

calcular_precio_con_iva()

Tipos de parámetros

Parámetros Formales: son variables locales declaradas en la definición de la función, que se utilizarán para recibir datos con los que va a operar la misma.

Parámetros Actuales: son las variables o datos que recibe la función al momento de ser invocada.



Parámetros por posición

La forma más intuitiva de trabajar con parámetros es pasandolos respetando la posición.

```
def restar(numero_a, numero_b):  
    diferencia = numero_a - numero_b  
    return diferencia  
  
print(restar(15, 5)) # 10
```

Parámetros por nombre

Otra forma de pasar parámetros a una función es asignando valores a las variables utilizando sus nombres.

```
def restar(numero_a, numero_b):  
    diferencia = numero_a - numero_b  
    return diferencia  
  
print(restar(numero_b = 5, numero_a = 15)) # 10
```


Parámetros opcionales

Python permite trabajar con parámetros opcionales o por defecto, estos deben ir siempre después de los obligatorios.

```
def restar(numero_a, numero_b = 5):  
    diferencia = numero_a - numero_b  
    return diferencia  
  
print(restar(15)) # 10  
print(restar(15, 7)) # 8
```

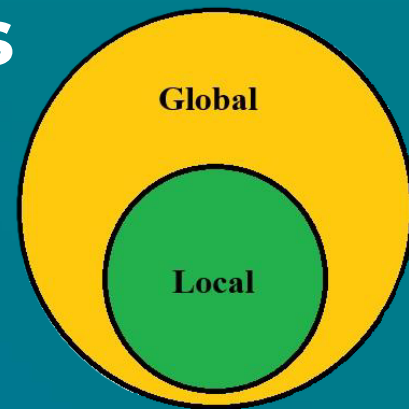
Parámetros y retorno con tipos

Se recomienda indicar que tipo de dato se espera que reciban los parámetros y que tipo de dato devuelve la función. * En Python estas indicaciones NO son restrictivas.

```
def restar(numero_a:int, numero_b:int = 5)-> int:  
    diferencia = numero_a - numero_b  
    return diferencia  
  
print(restar(15)) # 10
```

Variables locales y globales

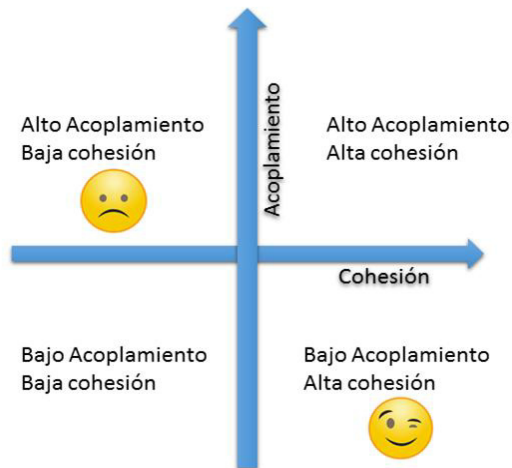
- Las variables **locales** son aquellas que se definen **dentro de una función** y solo son accesibles dentro de esa función.
- Las variables **globales** son aquellas que se definen **fuera de todas las funciones** y son accesibles desde cualquier parte del programa.



El **scope** (ámbito de la función) es donde las variables **locales** nacen y mueren, este mismo está separado por **sangría** al igual que las estructuras lógicas y repetitivas.

Nota: las variables globales, dentro de una función, pueden ser modificadas solo si se las marca como globales dentro del scope de la misma.

Cohesión y Acoplamiento :



Cohesión: Implica una conexión o unión que asegura que las partes de un todo estén bien integradas y funcionen juntas de manera efectiva.

Acoplamiento: Es la dependencia que tienen las funciones entre sí.

¿Qué es una Referencia?

Una referencia en programación es una forma de acceder a un valor mediante su **dirección de memoria** en lugar de interactuar directamente con el valor. Esto permite manipular el dato original sin crear copias adicionales, lo cual es eficiente en términos de recursos. Al utilizar una referencia, cualquier modificación realizada a través de ella afectará directamente al valor almacenado en esa dirección de memoria.

Pasaje por valor y por referencia en general :

En principio los conceptos de paso por **valor** y por **referencia** que aplican a la hora de **cómo** trata una función a los parámetros que se le pasan como entrada.

- Por VALOR: la función creará una copia local de la variable y cualquier modificación sobre ella no tendrá ningún efecto sobre la original.
- Por REFERENCIA: la función recibirá la dirección de memoria de la variable original y podrá realizar modificaciones directamente sobre ella.

Pasaje por valor y por referencia en Python :

En Python, **los parámetros de una función se pasan por referencia**, pero esta referencia es en realidad una referencia al objeto, no a la variable original. Esto significa que cuando se pasa una variable a una función, la función recibe una referencia al objeto al que apunta la variable.

Sin embargo, no puede modificar la variable original si el objeto al que apunta es **inmutable**, como números o cadenas de texto. En cambio, si el objeto es **mutable**, como una lista, un diccionario o una instancia de una clase, los cambios realizados dentro de la función afectarán al objeto original.

Pasaje por valor y por referencia

pass by reference



fillCup()

pass by value



fillCup()

Documentación

La documentación de una función es fundamental para la claridad, comprensión, y mantenibilidad del código, facilitando tanto el trabajo individual como la colaboración en equipos de desarrollo.

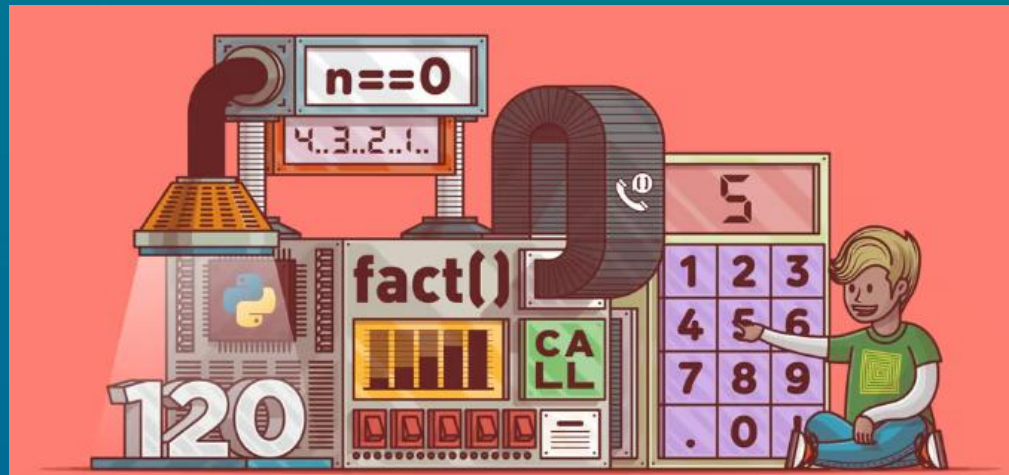
```
def sumar(numero_a:int, numero_b:int) -> int:
    """Que debe hacer la función
    Args:
        Que parametros recibe
    Returns:
        Que retorna
    """
    suma = numero_a + numero_b
    return suma
```

```
def sumar(numero_a:int, numero_b:int) -> int:
    """Funcion que se encarga de sumar dos
    números enteros

    Args:
        numero_a (int): Primer número
        numero_b (int): Segundo Número

    Returns:
        int: Devuelve la suma entre estos dos
        números
    """
    suma = numero_a + numero_b
    return suma
```

Funciones recursivas



Las muñecas rusas (matrioshkas)

Las matrioshkas son un conjunto de muñecas que se colocan una dentro de la otra. Cada muñeca se abre para revelar otra más pequeña en su interior, y así sucesivamente, hasta llegar a la más pequeña que no se abre.

A partir de esta analogía, podemos analizar el concepto de **recursividad**.



Recursividad

Cuando un proceso se define en términos de sí mismo hablamos de recursividad. Básicamente un problema podrá ser resuelto de forma recursiva si la solución se puede expresar en términos de si misma. Para poder obtener esta solución, deberá resolverse el mismo problema sobre un conjunto de datos de entrada menor.

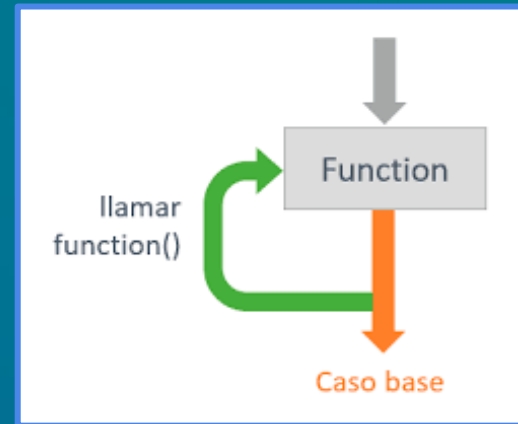
¿En qué trabajás? Estoy intentando arreglar los problemas que creé cuando intentaba arreglar los problemas que creé cuando intentaba arreglar los problemas que creé.

Y así nació la recursividad.



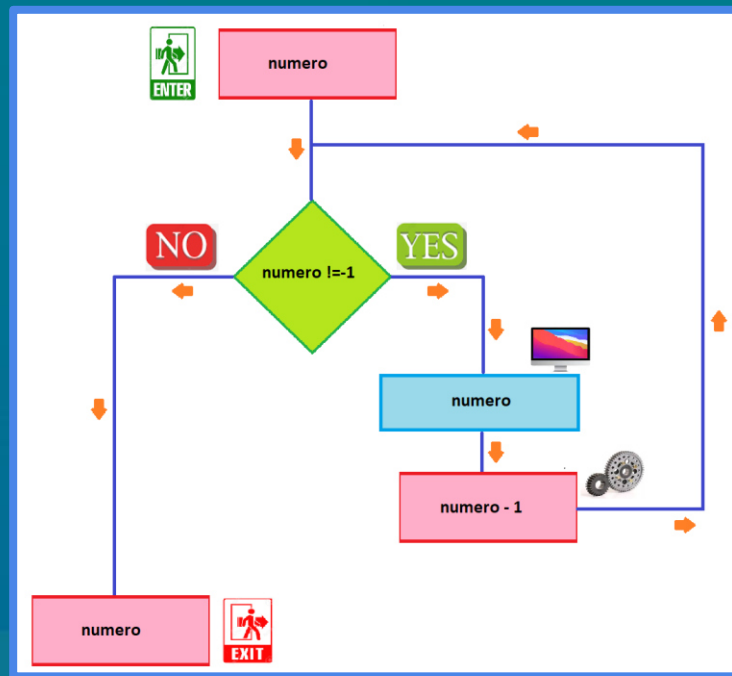
Funciones recursivas

La recursividad aplicada al mundo de la programación nos permite resolver tareas en donde las mismas pueden ser divididas en **sub tareas cuya funcionalidad es la misma**. Una función recursiva es aquella que está definida en función de sí misma, por lo que se llama repetidamente a sí misma **hasta llegar a un punto de salida**.



Pongámonos a prueba

El siguiente diagrama de flujo muestra una cuenta regresiva desde un número ingresado hasta el 0. Planteemos una solución desde un punto de vista convencional (con una estructura repetitiva) y desde un punto de vista recursivo



Factorial de un número

El factorial de un número entero positivo se define como el producto de todos los números enteros positivos desde la unidad hasta el número definido.

Matemáticamente definimos al factorial:

$$n! = n * (n - 1)!$$

Si $n \geq 0$



$$5! = 5 * 4 * 3 * 2 * 1 = 5 * 4! = 120$$

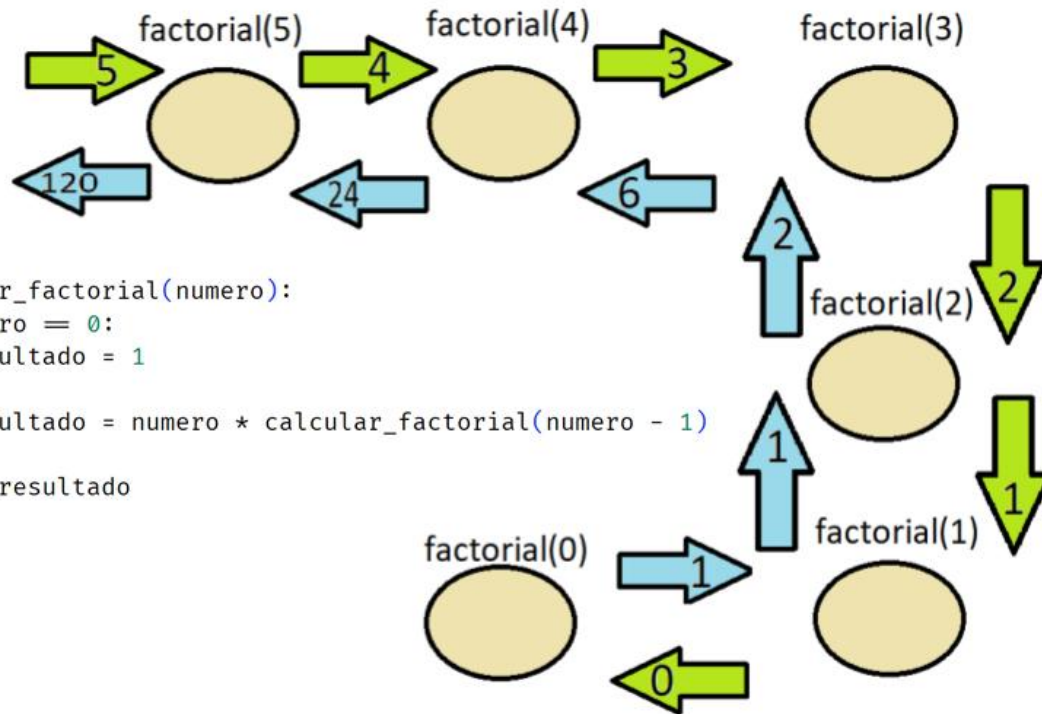
$$4! = 4 * 3 * 2 * 1 = 4 * 3! = 24$$

$$3! = 3 * 2 * 1 = 3 * 2! = 6$$

$$2! = 2 * 1 = 2 * 1! = 2$$

$$1! = 1$$

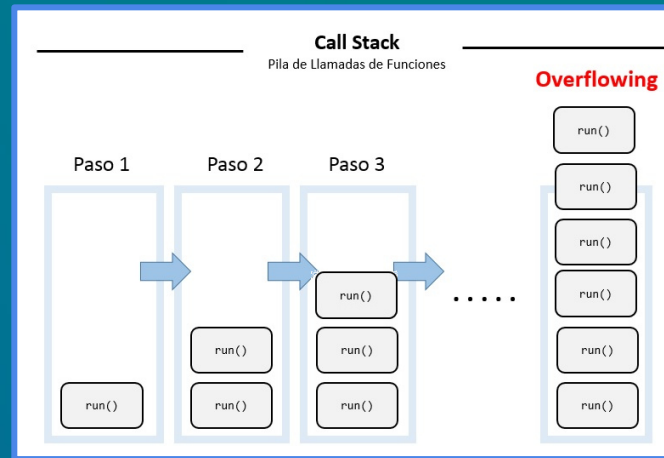
$$0! = 1$$



```
def calcular_factorial(numero):
    if numero == 0:
        resultado = 1
    else:
        resultado = numero * calcular_factorial(numero - 1)
    return resultado
```

Pero ... ¿qué pasa en memoria?

- La gestión de memoria en las funciones recursivas se realiza mediante la pila de llamadas.
- Cada vez que se llama a una función recursiva, se agrega una nueva instancia de esa función a la pila de llamadas.
- A medida que la función se llama recursivamente se acumulan mas y mas instancias en la pila.
- Cuando se alcanza la condición de salida, las funciones comienzan a desapilarse, en caso de que dicha condición no se cumpla y se llegue a cierto límite (llamado el límite de recursión), python lanzará un error de desbordamiento de pila (**RecursionError**).



Ventajas



- **Simplicidad conceptual:** algunos problemas se pueden expresar de manera más clara y concisa.
- **Solución elegante para problemas recursivos:** hay problemas que naturalmente se prestan a una solución recursiva, como árboles o la división y conquista.
- **Facilidad de mantenimiento:** en algunos casos, el código recursivo puede ser más fácil de entender y mantener que su equivalente repetitivo.

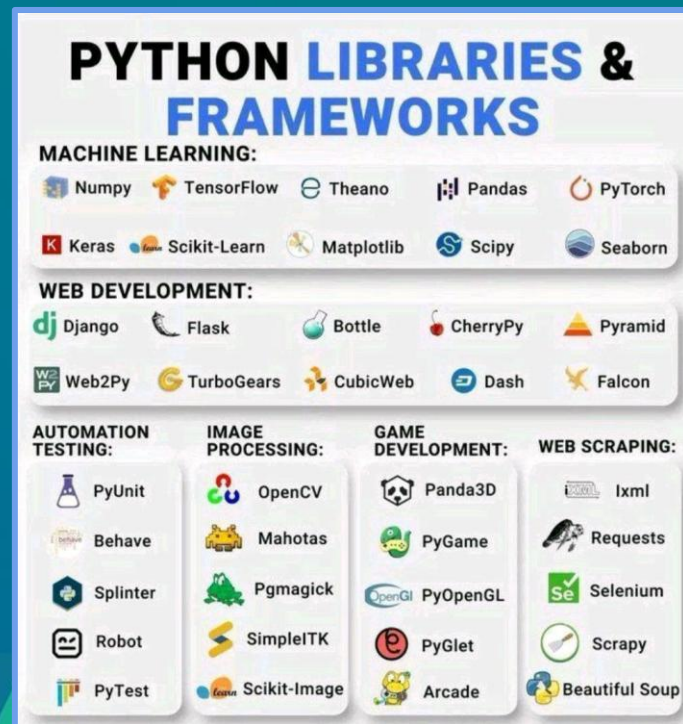
Desventajas



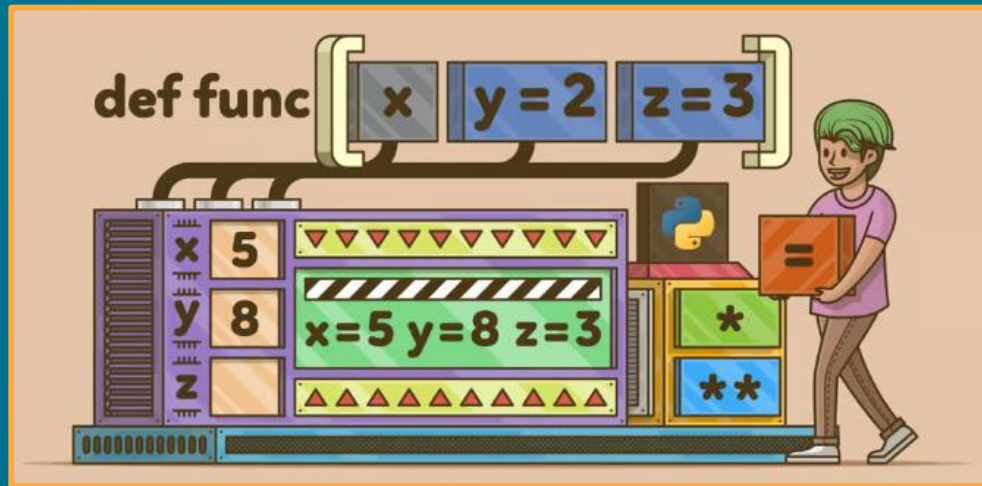
- **Consumo de memoria:** cada llamada recursiva agrega una nueva instancia de la función a la pila de llamadas, esto puede consumir una cantidad significativa de memoria, especialmente para problemas con profundidad recursiva.
- **Eficiencia:** en general las funciones recursivas pueden ser menos eficientes que sus equivalentes iterativos debido al costo adicional de gestionar la pila de llamadas y la posibilidad de realizar cálculos redundantes.
- **Límite de recursión:** python tiene un límite en la profundidad de recursión, por lo que las funciones recursivas pueden no ser adecuadas para problemas con grandes profundidades recursivas.

Módulos de terceros

Es inmensurable la cantidad de módulos y paquetes desarrollados por la comunidad de python, cada uno de ellos con propósitos específicos.



Funciones



Introducción

En Python, una función es un bloque de código que realiza una tarea específica.

¿Qué es una función?

A medida que los programas crecen en extensión y complejidad la resolución se torna más **complicada** y su **depuración** y **modificaciones** futuras resultan casi imposibles. Para resolver este tipo de problemas lo que se hace es **dividir el programa** en módulos más pequeños que cumplan **una tarea simple y bien acotada** llamados funciones.

¿Para qué sirve una función?

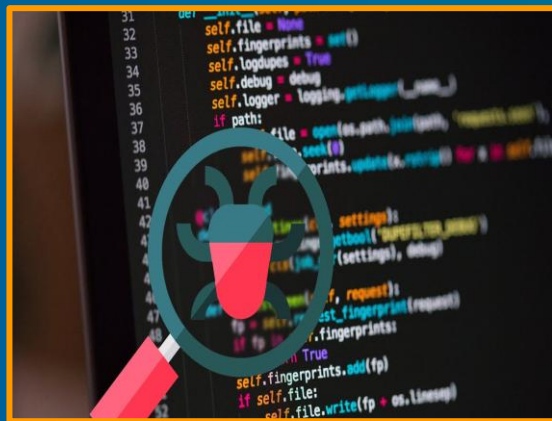
- Las funciones sirven para **descomponer una tarea específica en un bloque de código** que se puede llamar varias veces desde cualquier parte del programa.
- Las funciones permiten que el código sea **modular, legible y fácil de mantener**.

Objetivos de una función:



Minificación: Logramos que el programa sea más simple de comprender ya que cada función se dedica a realizar una tarea en particular.

Objetivos de una función:



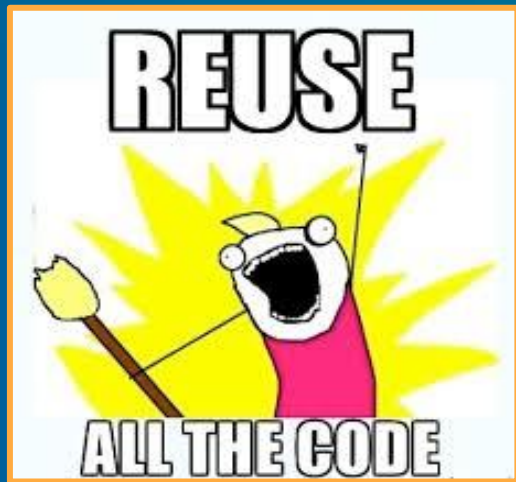
Depuración: La depuración queda acotada a cada función.

Objetivos de una función:



Modificación: Las modificaciones al programa se reducen a cada módulo.

Objetivos de una función:



Reutilización: Cuando cada función está bien probada, se la puede usar las veces que se quiera y así reutilizar código.

Objetivos de una función:



Independencia: Se obtiene una autonomía del código donde cada función es independiente de otra.

Componentes de una función:

Palabra reservada

Nombre

Parámetro

Parámetro opcional

```
def calcular_precio_con_iva(valor_sin_iva, iva=21):
```

```
    '''
```

```
    Documentacion
```

```
    '''
```

```
    resultado = valor_sin_iva * (1 + (iva / 100))
```

```
    return resultado
```

Documentación

Variable local

Valor del retorno

Desarrollo e invocación

```
def calcular_precio_con_iva(valor_sin_iva, iva = 21):  
    resultado = valor_sin_iva * (1 + iva / 100)  
    return resultado
```

Definición y
desarrollo

```
print(calcular_precio_con_iva(100))    # 121.0  
print(calcular_precio_con_iva(100, 10.5)) # 110.5
```

Invocación o
llamada

Palabras reservadas: **def** y **return**

El uso de **def** nos permite crear una función.
Podemos utilizar **return** para devolver un valor.

```
def sumar(numero_a, numero_b):  
    suma = numero_a + numero_b  
    return suma  
  
resultado = sumar(33, 2)  
print("El resultado de la suma es", resultado)
```

Nombre

Las funciones y las variables se deberán escribir en formato **snake_case**.

Las palabras separadas entre sí por barra baja (underscore) en vez de espacios y las letras en minúscula.

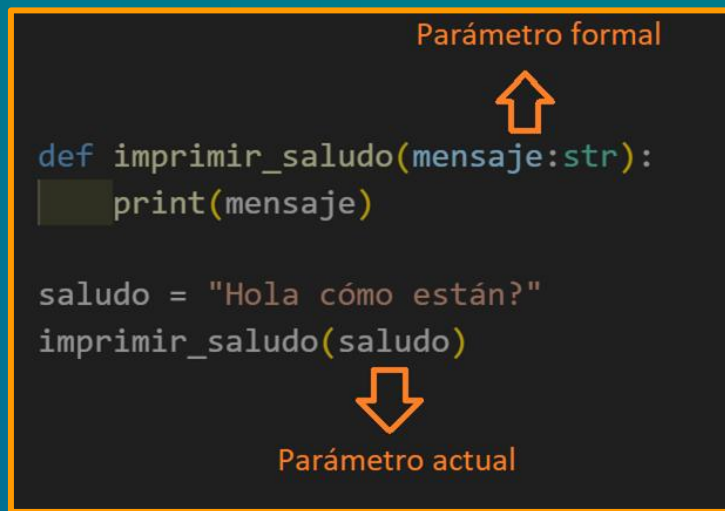
Las funciones representan una acción, por lo tanto, el nombre debe contener un verbo.

calcular_precio_con_iva()

Tipos de parámetros

Parámetros Formales: son variables locales declaradas en la definición de la función, que se utilizarán para recibir datos con los que va a operar la misma.

Parámetros Actuales: son las variables o datos que recibe la función al momento de ser invocada.



Parámetros por posición

La forma más intuitiva de trabajar con parámetros es pasándolos respetando la posición.

```
def restar(numero_a, numero_b):  
    diferencia = numero_a - numero_b  
    return diferencia  
  
print(restar(15, 5)) # 10
```

Parámetros por nombre

Otra forma de pasar parámetros a una función es asignando valores a las variables utilizando sus nombres.

```
def restar(numero_a, numero_b):  
    diferencia = numero_a - numero_b  
    return diferencia  
  
print(restar(numero_b = 5, numero_a = 15)) # 10
```

Parámetros opcionales

Python permite trabajar con parámetros opcionales o por defecto, estos deben ir siempre después de los obligatorios.

```
def restar(numero_a, numero_b = 5):  
    diferencia = numero_a - numero_b  
    return diferencia  
  
print(restar(15)) # 10  
print(restar(15, 7)) # 8
```

Parámetros y retorno con tipos

Se recomienda indicar que tipo de dato se espera que reciban los parámetros y que tipo de dato devuelve la función. * En Python estas indicaciones NO son restrictivas.

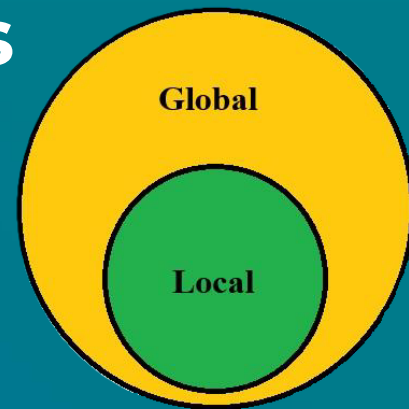
```
def restar(numero_a:int, numero_b:int = 5)-> int:  
    diferencia = numero_a - numero_b  
    return diferencia  
  
print(restar(15)) # 10
```


Variables locales y globales

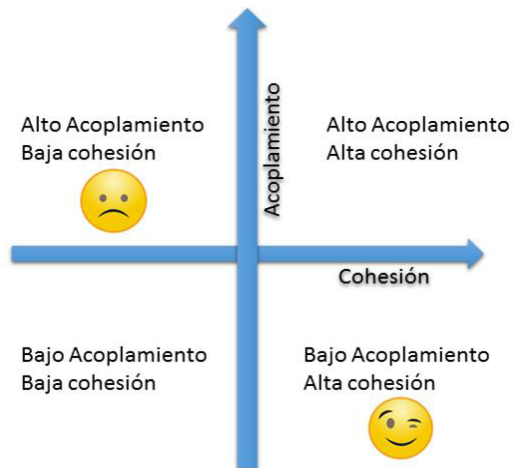
- Las variables **locales** son aquellas que se definen **dentro de una función** y solo son accesibles dentro de esa función.
- Las variables **globales** son aquellas que se definen **fuera de todas las funciones** y son accesibles desde cualquier parte del programa.

El **scope** (ámbito de la función) es donde las variables **locales** nacen y mueren, este mismo está separado por **sangría** al igual que las estructuras lógicas y repetitivas.

Nota: las variables globales, dentro de una función, pueden ser modificadas solo si se las marca como globales dentro del scope de la misma.



Cohesión y Acoplamiento :



Cohesión: Implica una conexión o unión que asegura que las partes de un todo estén bien integradas y funcionen juntas de manera efectiva.

Acoplamiento: Es la dependencia que tienen las funciones entre sí.

¿Qué es una Referencia?

Una referencia en programación es una forma de acceder a un valor mediante su **dirección de memoria** en lugar de interactuar directamente con el valor. Esto permite manipular el dato original sin crear copias adicionales, lo cual es eficiente en términos de recursos. Al utilizar una referencia, cualquier modificación realizada a través de ella afectará directamente al valor almacenado en esa dirección de memoria.

Pasaje por valor y por referencia en general :

En principio los conceptos de paso por **valor** y por **referencia** que aplican a la hora de **cómo** trata una función a los parámetros que se le pasan como entrada.

- Por VALOR: la función creará una copia local de la variable y cualquier modificación sobre ella no tendrá ningún efecto sobre la original.
- Por REFERENCIA: la función recibirá la dirección de memoria de la variable original y podrá realizar modificaciones directamente sobre ella.

Pasaje por valor y por referencia en Python :

En Python, **los parámetros de una función se pasan por referencia**, pero esta referencia es en realidad una referencia al objeto, no a la variable original. Esto significa que cuando se pasa una variable a una función, la función recibe una referencia al objeto al que apunta la variable.

Sin embargo, no puede modificar la variable original si el objeto al que apunta es **inmutable**, como números o cadenas de texto. En cambio, si el objeto es **mutable**, como una lista, un diccionario o una instancia de una clase, los cambios realizados dentro de la función afectarán al objeto original.

Pasaje por valor y por referencia

pass by reference



fillCup()

pass by value



fillCup()

Documentación

La documentación de una función es fundamental para la claridad, comprensión, y mantenibilidad del código, facilitando tanto el trabajo individual como la colaboración en equipos de desarrollo.

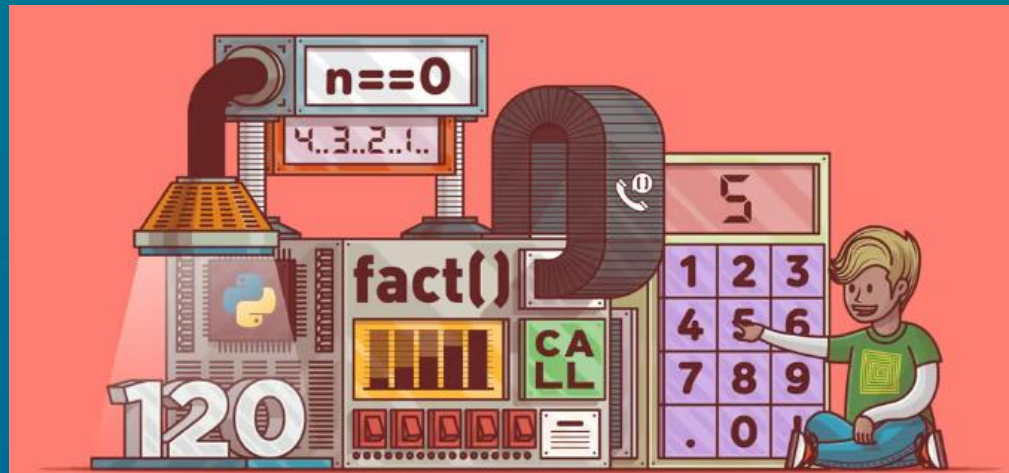
```
def sumar(numero_a:int, numero_b:int) -> int:
    """Que debe hacer la función
    Args:
        Que parametros recibe
    Returns:
        Que retorna
    """
    suma = numero_a + numero_b
    return suma
```

```
def sumar(numero_a:int, numero_b:int) -> int:
    """Funcion que se encarga de sumar dos
    números enteros

    Args:
        numero_a (int): Primer número
        numero_b (int): Segundo Número

    Returns:
        int: Devuelve la suma entre estos dos
        números
    """
    suma = numero_a + numero_b
    return suma
```


Funciones recursivas



Las muñecas rusas (matrioshkas)

Las matrioshkas son un conjunto de muñecas que se colocan una dentro de la otra. Cada muñeca se abre para revelar otra más pequeña en su interior, y así sucesivamente, hasta llegar a la más pequeña que no se abre.

A partir de esta analogía, podemos analizar el concepto de **recursividad**.



Recursividad

Cuando un proceso se define en términos de sí mismo hablamos de recursividad. Básicamente un problema podrá ser resuelto de forma recursiva si la solución se puede expresar en términos de si misma. Para poder obtener esta solución, deberá resolverse el mismo problema sobre un conjunto de datos de entrada menor.

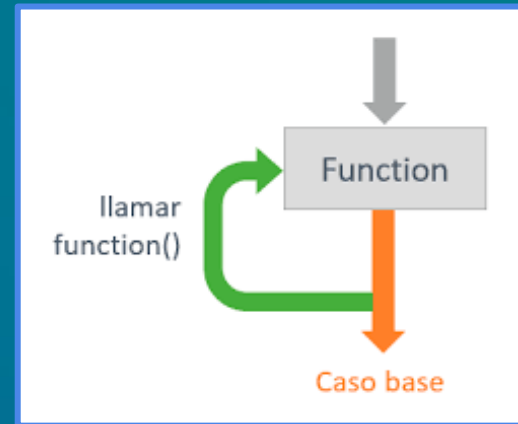
¿En qué trabajás? Estoy intentando arreglar los problemas que creé cuando intentaba arreglar los problemas que creé cuando intentaba arreglar los problemas que creé.

Y así nació la recursividad.



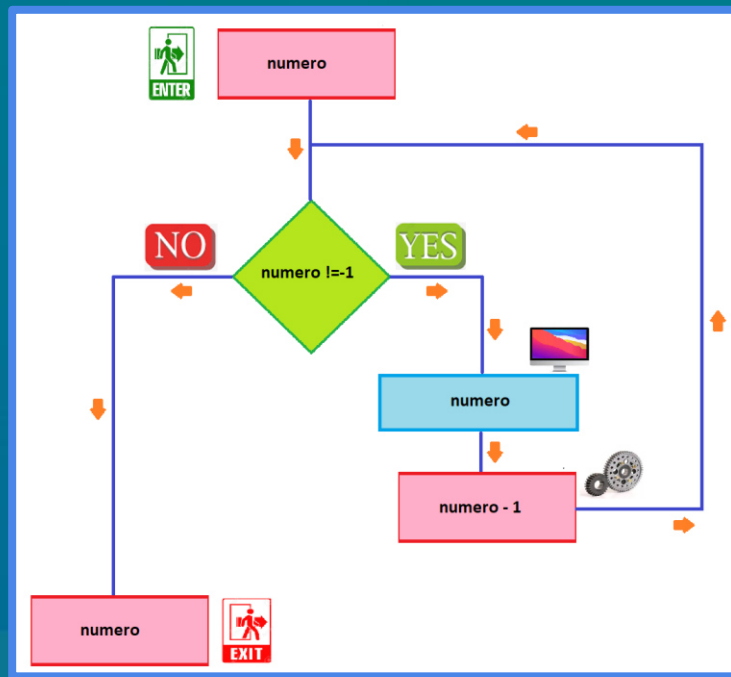
Funciones recursivas

La recursividad aplicada al mundo de la programación nos permite resolver tareas en donde las mismas pueden ser divididas en **sub tareas cuya funcionalidad es la misma**. Una función recursiva es aquella que está definida en función de sí misma, por lo que se llama repetidamente a sí misma **hasta llegar a un punto de salida**.



Pongámonos a prueba

El siguiente diagrama de flujo muestra una cuenta regresiva desde un número ingresado hasta el 0. Planteemos una solución desde un punto de vista convencional (con una estructura repetitiva) y desde un punto de vista recursivo



Factorial de un número

El factorial de un número entero positivo se define como el producto de todos los números enteros positivos desde la unidad hasta el número definido.

Matemáticamente definimos al factorial:

$$n! = n * (n - 1)!$$

Si $n \geq 0$



$$5! = 5 * 4 * 3 * 2 * 1 = 5 * 4! = 120$$

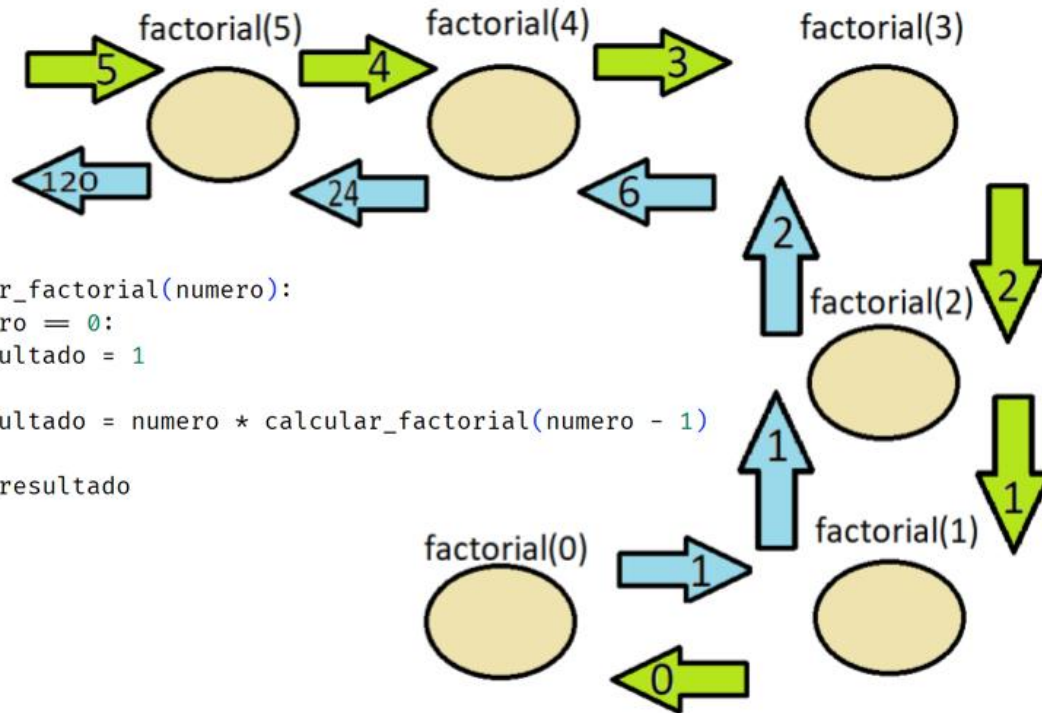
$$4! = 4 * 3 * 2 * 1 = 4 * 3! = 24$$

$$3! = 3 * 2 * 1 = 3 * 2! = 6$$

$$2! = 2 * 1 = 2 * 1! = 2$$

$$1! = 1$$

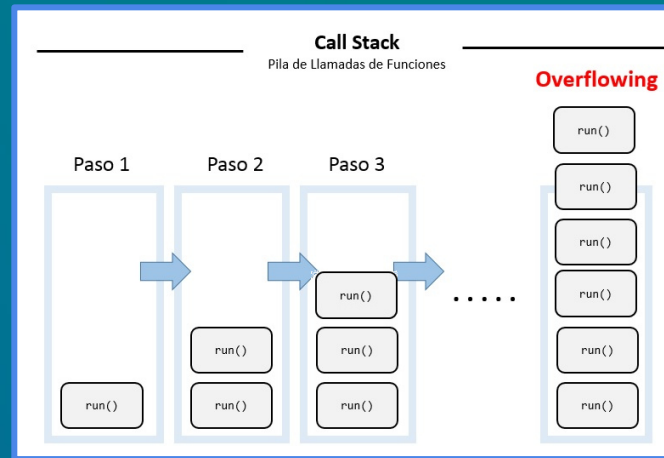
$$0! = 1$$



```
def calcular_factorial(numero):
    if numero == 0:
        resultado = 1
    else:
        resultado = numero * calcular_factorial(numero - 1)
    return resultado
```


Pero ... ¿qué pasa en memoria?

- La gestión de memoria en las funciones recursivas se realiza mediante la pila de llamadas.
- Cada vez que se llama a una función recursiva, se agrega una nueva instancia de esa función a la pila de llamadas.
- A medida que la función se llama recursivamente se acumulan mas y mas instancias en la pila.
- Cuando se alcanza la condición de salida, las funciones comienzan a desapilarse, en caso de que dicha condición no se cumpla y se llegue a cierto límite (llamado el límite de recursión), python lanzará un error de desbordamiento de pila (**RecursionError**).



Ventajas



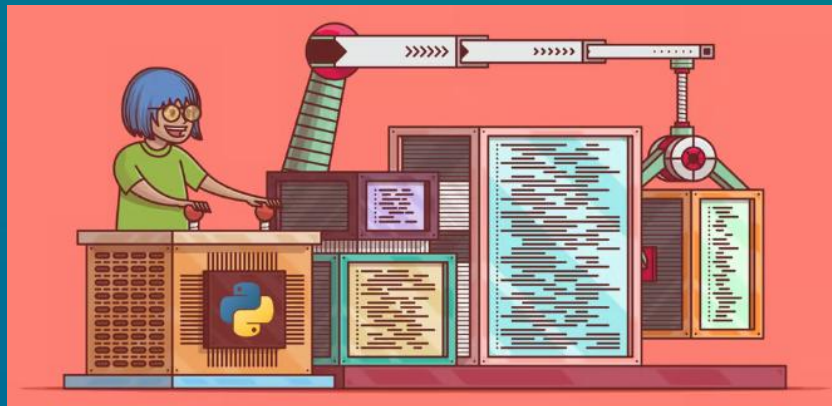
- **Simplicidad conceptual:** algunos problemas se pueden expresar de manera más clara y concisa.
- **Solución elegante para problemas recursivos:** hay problemas que naturalmente se prestan a una solución recursiva, como árboles o la división y conquista.
- **Facilidad de mantenimiento:** en algunos casos, el código recursivo puede ser más fácil de entender y mantener que su equivalente repetitivo.

Desventajas



- **Consumo de memoria:** cada llamada recursiva agrega una nueva instancia de la función a la pila de llamadas, esto puede consumir una cantidad significativa de memoria, especialmente para problemas con profundidad recursiva.
- **Eficiencia:** en general las funciones recursivas pueden ser menos eficientes que sus equivalentes iterativos debido al costo adicional de gestionar la pila de llamadas y la posibilidad de realizar cálculos redundantes.
- **Límite de recursión:** python tiene un límite en la profundidad de recursión, por lo que las funciones recursivas pueden no ser adecuadas para problemas con grandes profundidades recursivas.

Módulos y paquetes



Módulos

A medida que los programas crecen, es necesario separarlo en varios archivos para que el mantenimiento sea más sencillo. Quizás también sea necesario usar una función ya escrita en otro programa sin la necesidad de copiar su definición en cada programa que la quiera utilizar. Para ello, los módulos son la solución.

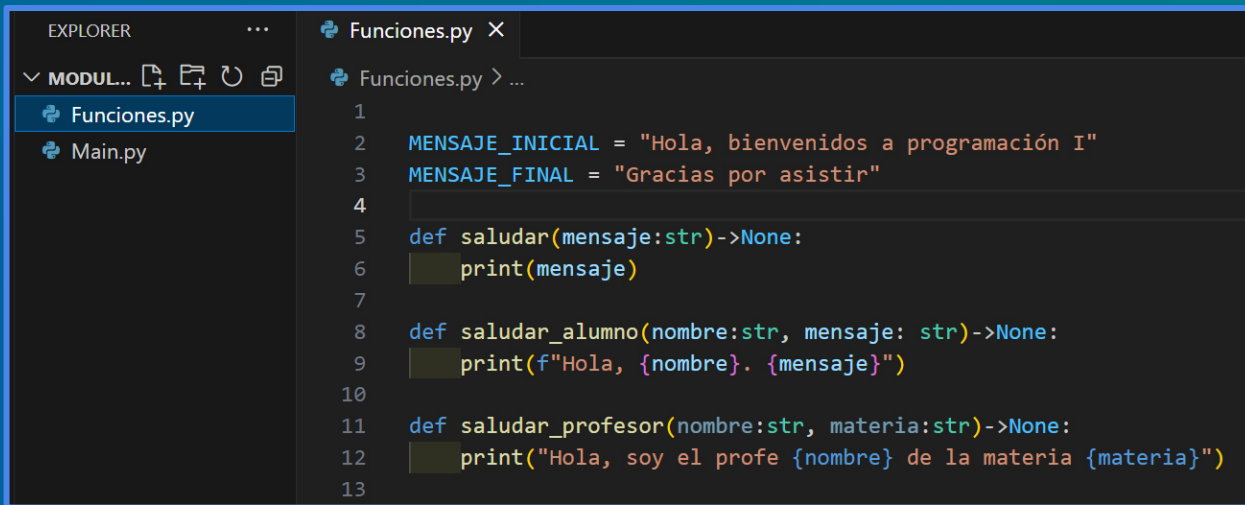


Generalidades

- Un módulo en Python es un archivo que contiene definiciones de funciones, clases y variables, junto con cualquier otro código ejecutable.
- Tienen la extensión .py.
- Pueden ser importados y utilizados en otros programas de Python.
- Ayudan a organizar y reutilizar el código al dividirlo en componentes más manejables y específicos.

Creación

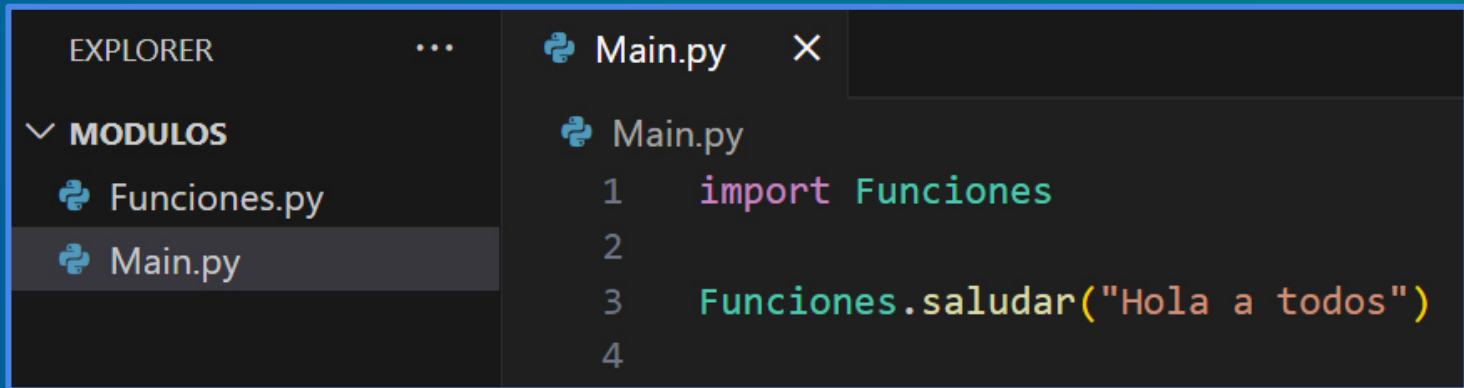
Supongamos que definimos las siguientes funciones y variables para saludar en un archivo .py (en este caso nuestro módulo)



```
1
2 MENSAJE_INICIAL = "Hola, bienvenidos a programación I"
3 MENSAJE_FINAL = "Gracias por asistir"
4
5 def saludar(mensaje:str)->None:
6     print(mensaje)
7
8 def saludar_alumno(nombre:str, mensaje: str)->None:
9     print(f"Hola, {nombre}. {mensaje}")
10
11 def saludar_profesor(nombre:str, materia:str)->None:
12     print("Hola, soy el profe {nombre} de la materia {materia}")
13
```

Importación

Para poder utilizar las funciones definidas en el módulo debemos importar dicho módulo en el archivo en donde queremos utilizar las funciones:



The screenshot shows a code editor with a dark theme. On the left, the 'EXPLORER' sidebar is open, showing a folder named 'MODULOS' containing two files: 'Funciones.py' and 'Main.py'. 'Main.py' is selected. The main editor area shows the code in 'Main.py':

```
1 import Funciones
2
3 Funciones.saludar("Hola a todos")
4
```

En este caso, para utilizar cualquiera de las funciones definidas en el módulo, tenemos que anteponer el nombre del mismo antes de llamar a la función, seguido del operador punto.

Importación

Observemos las siguientes alternativas para importar módulos y sus componentes:

```
Main.py X
Main.py
1  from Funciones import saludar
2
3  saludar("Hola a todos")
4
```

Caso 1: desde el módulo *Funciones*, únicamente importamos la función *saludar*.

```
Main.py X
Main.py
1  from Funciones import saludar, saludar_alumno
2
3  saludar("Estamos probando modulos")
4
5  saludar_alumno("Pepe", "Tenes que estudiar mas!!!")
6
```

Caso 2: desde el módulo *Funciones*, importamos las funciones *saludar* y *saludar_alumno*.

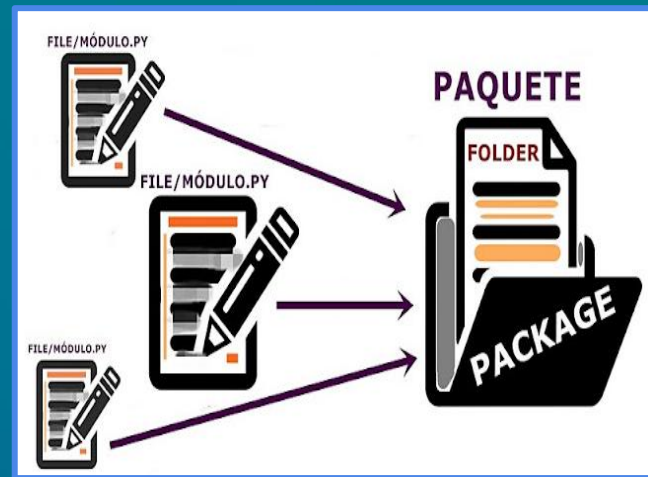
```
Main.py X
Main.py
1  from Funciones import *
2
3  saludar(MENSAJE_INICIAL)
4
5  saludar_profesor("Juan", "Química")
6
7  saludar_alumno("Pepe", "Tenes que estudiar mas!!!")
8
9  saludar(MENSAJE_FINAL)
```

Caso 3: desde el módulo *Funciones*, importamos todas las funciones y variables globales con el operador ***.

Paquetes

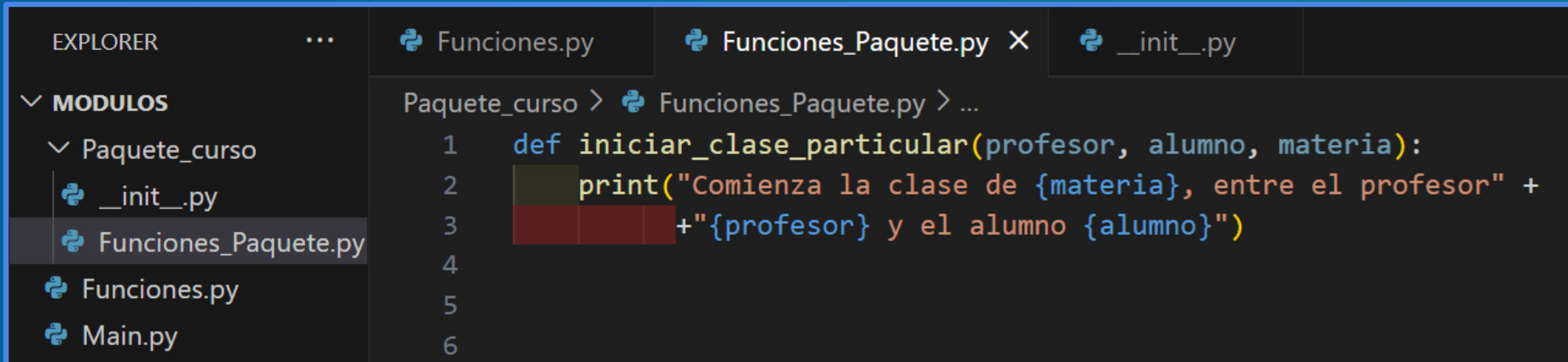
Un paquete en Python es una forma de estructurar y organizar módulos relacionados.

Permiten reutilizar código de manera más efectiva, al agrupar funcionalidades relacionadas en un solo lugar. Esto facilita la gestión de proyectos grandes y complejos al proporcionar una estructura jerárquica y modular para el código.



Creación

Para crear un paquete debemos crear una carpeta junto con un archivo `__init__.py` (de esta manera el intérprete de python entenderá que la carpeta se trata de un paquete). Luego agregamos en el mismo los módulos que necesitemos agrupar.



```
EXPLORER  ...  Funciones.py  Funciones_Paquete.py X  __init__.py

▼ MODULOS
  ▼ Paquete_curso
    __init__.py
    Funciones_Paquete.py
    Funciones.py
    Main.py

Paquete_curso > Funciones_Paquete.py > ...
1  def iniciar_clase_particular(profesor, alumno, materia):
2      print("Comienza la clase de {materia}, entre el profesor" +
3          "{profesor} y el alumno {alumno}")
4
5
6
```

Importación

De la misma forma que importamos módulos independientes, podremos importar el/los módulos necesarios que se encuentran en un paquete.

```
Main.py  X
Main.py
1  #UTILIZAMOS UN ALIAS
2  import Paquete_curso.Funciones_Paquete as PC
3
4  PC.iniciar_clase_particular(profesor = "Pedro", alumno = "Juan", materia = "Matemática")
5
```

```
Main.py  X
Main.py
1  from Funciones import * # Importando todas las funciones del módulo Funciones
2
3  # Importando la funcion iniciar_clase_particular del modulo Funciones_Paquete
4  # del paquete Paquete_curso
5  from Paquete_curso.Funciones_Paquete import iniciar_clase_particular
6
7  iniciar_clase_particular(profesor = "Pedro", alumno = "Juan", materia = "Matemática")
```

Comunicar módulos

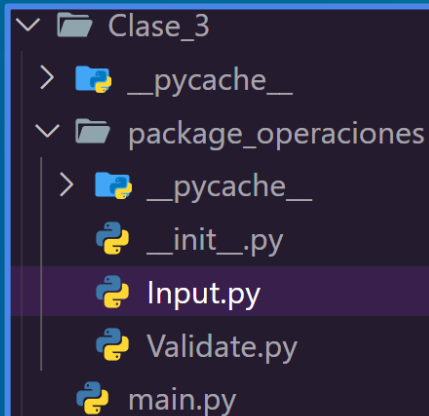
Si necesitamos comunicar 2 módulos de un mismo paquete como por ejemplo Input y Validate:

Desde Input deberemos importar Validate de la siguiente manera.

```
from .Validate import *
```

Esto se debe a que Python utiliza la convención de puntos para organizar los módulos en paquetes.

Comunicar módulos



```
1 from .Validate import * # Manera de importar entre módulos
2
3 def get_int():
4     validate_int() # Función de el módulo Validate
```

package

module

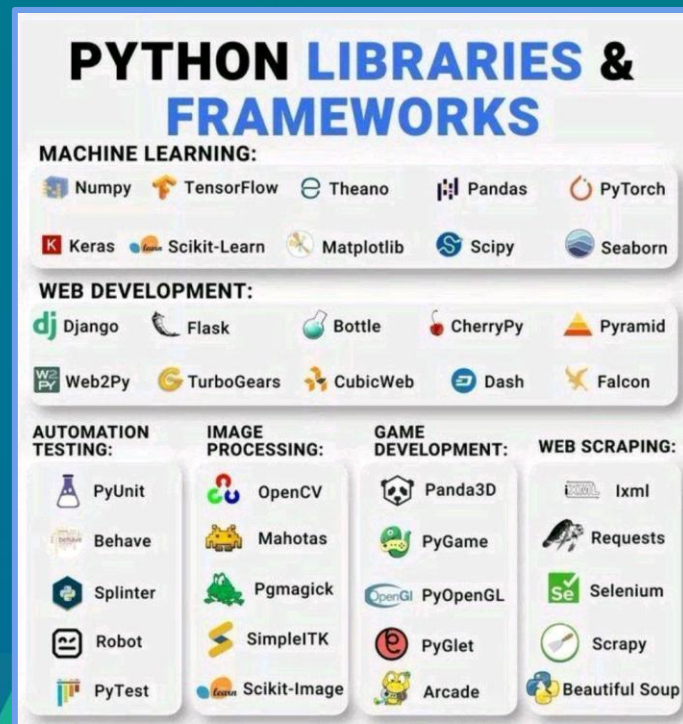
function

Son un conjunto de módulos y paquetes que se distribuyen junto con Python. Muchas de las operaciones más comunes ya se implementan en ellas.



Módulos de terceros

Es inmensurable la cantidad de módulos y paquetes desarrollados por la comunidad de python, cada uno de ellos con propósitos específicos.



Instalando módulos de terceros

Cada uno de estos paquetes está identificado por su nombre. Para instalar alguno de ellos utilizaremos una herramienta llamada pip que se incluye con la instalación de Python.

```
pip install paquete
```

En caso de que tu SO no reconozca pip:

```
python -m pip install paquete
```



```
py -m pip install paquete
```

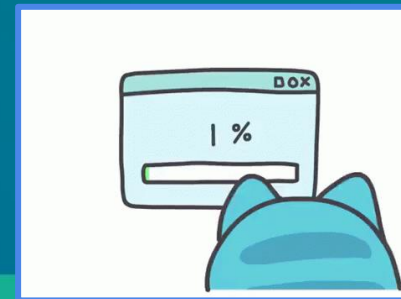


[Si no pudiste solucionarlo, consulta esta página](#)



Problemas instalando el módulo pygame
Para asegurarnos que funciona correctamente,
corramos el siguiente comando:

```
[python | py] -m pygame.examples.aliens
```

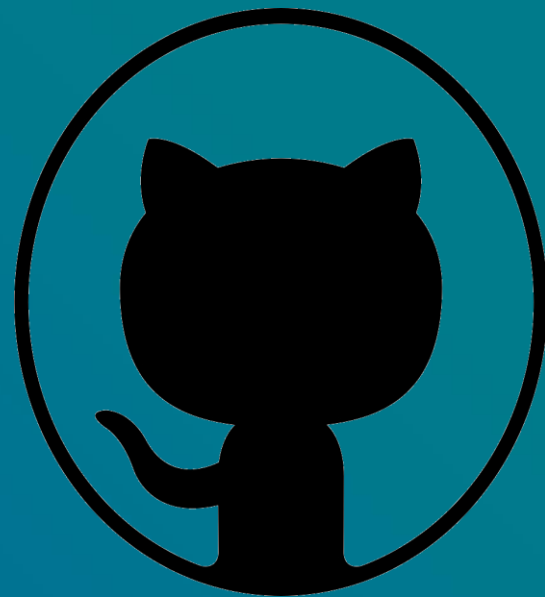


Control de versiones



¿Qué es GitHub?

GitHub es una plataforma de desarrollo colaborativo de software que permite a los desarrolladores alojar proyectos en repositorios remotos y colaborar con otros a través de un control de versiones basado en Git.

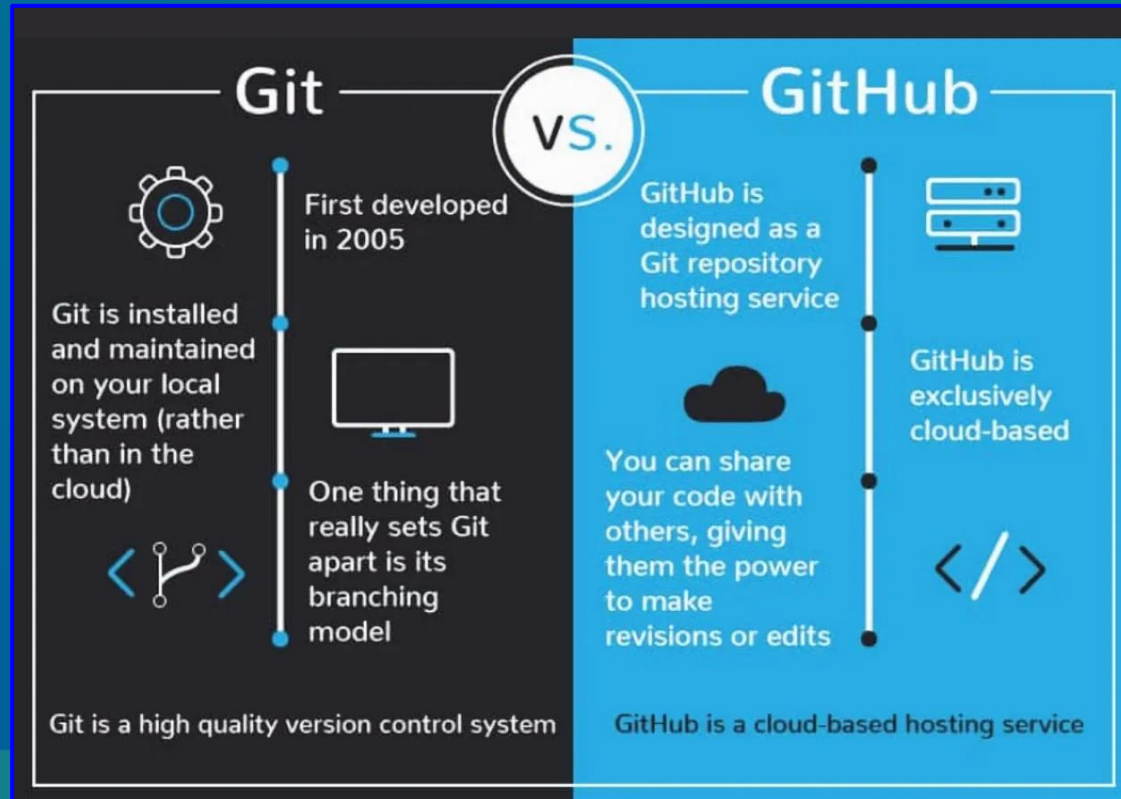




¿Qué es Git?

Git es un sistema de control de versiones open source que permite a los desarrolladores realizar un seguimiento de los cambios en el código fuente a lo largo del tiempo, facilitando la colaboración y la gestión de proyectos de software.





Metodología de trabajo

1. Crear una cuenta en www.github.com
2. Descargar e instalar git desde <https://git-scm.com/downloads>
3. Configurar git:
Abrir una terminal y configurar nombre y correo electrónico globalmente.

```
git config --global user.name "Tu Nombre"  
git config --global user.email "tuemail@example.com"
```

Comandos git

La tecnología git es muy amplia. Existen infinidad de comandos para poder trabajar de manera local y remota con un repositorio. Muchos de esos comandos los aprenderán a lo largo de la carrera. Aquí va una lista de los comandos básicos que usaremos en esta cátedra.

git init

Inicializa un nuevo repositorio de Git en el directorio actual, creando una carpeta .git.

Nota: para inicializar un repositorio previamente tenemos que crear una carpeta y abrir la misma desde línea de comandos.

```
Initialized empty Git repository in /ruta/a/tu/proyecto/.git/
```

git clone

Me permite descargar y guardar una copia del repositorio remoto de github en nuestro repositorio local de git.

`git clone "url del repositorio remoto"`

git status

Muestra el estado actual del repositorio, incluyendo los archivos modificados, agregados, y no rastreados (a modo de ejemplo crear un archivo ejemplo.txt dentro de nuestra carpeta donde ejecutamos el comando init).

```
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    ejemplo.txt

nothing added to commit but untracked files present (use "git add" to track)
```

git add .

Agrega todos los archivos modificados en el directorio actual al área de preparación (staging area).

```
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   ejemplo.txt
```

Nota: luego de ejecutar nuevamente el comando status, git nos informa que hay cambios que están listos para ser impactados (committed)

git commit -m "mensaje"

Captura de Cambios: Git toma todos los cambios que han sido añadidos al área de preparación (staging area) mediante el comando `git add` y los guarda en el repositorio local como un nuevo commit. Un commit es un "punto de control" en el historial del proyecto, que registra el estado exacto de todos los archivos en ese momento y nos permite agregar un mensaje descriptivo del mismo.

```
git commit -m "Añadir ejemplo.txt al repositorio"
```

```
[master (root-commit) abc1234] Añadir ejemplo.txt al repositorio  
1 file changed, 1 insertion(+)  
create mode 100644 ejemplo.txt
```

git commit

Configuración variables globales

```
Usuario@GCX-PC MINGW64 ~/Desktop/Repositorio/9-9_111 (main)
$ git commit -m "Este es el primer commit"
Author identity unknown

*** Please tell me who you are.

Run

git config --global user.email "you@example.com"
git config --global user.name "Your Name"

to set your account's default identity.
Omit --global to set the identity only in this repository.

fatal: unable to auto-detect email address (got 'Usuario@GCX-PC.(none)')
```



git remote add origin <url del repositorio>

Asocia tu repositorio local con el repositorio remoto en GitHub.

```
git remote add origin https://github.com/tu_usuario/repo_prueba.git
```

Nota: verificar en github que el repositorio local se haya asociado al remoto.

git push

Envía los commits locales al repositorio remoto, actualizando el proyecto compartido.

```
Enumerating objects: 3, done.  
Counting objects: 100% (3/3), done.  
Writing objects: 100% (3/3), 217 bytes | 217.00 KiB/s, done.  
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0  
To https://github.com/tu_usuario/repo_prueba.git  
* [new branch]      master -> master  
Branch 'master' set up to track remote branch 'master' from 'origin'.
```



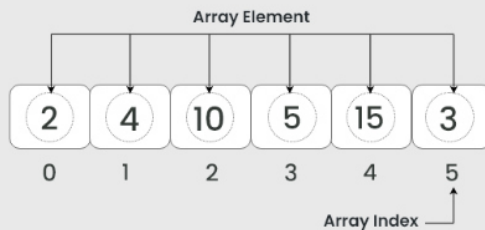
Arreglos unidimensionales: Vectores



¿Qué es un vector?

Un **arreglo unidimensional**, también conocido como **vector**, es una **estructura de datos que organiza elementos de manera lineal**, es decir, dispuestos uno tras otro en una sola dimensión. Cada elemento tiene una posición única y puede ser accedido utilizando un único índice.

Array
Data Structure



Beneficios del uso de vectores/listas :

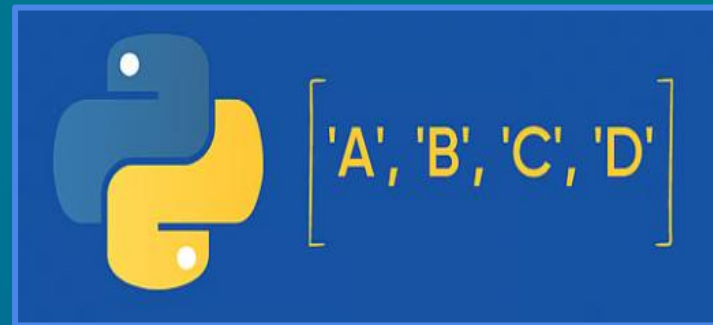
- **Organización y gestión de datos:** Las listas permiten agrupar datos relacionados en una sola estructura, simplificando su manejo y organización.
- **Acceso y manipulación eficiente:** Las listas facilitan el acceso, la iteración y la modificación de datos mediante índices, lo que es más eficiente que manejar variables individuales.
- **Escalabilidad:** Las listas son flexibles y escalables, permitiendo ajustar la cantidad de datos sin necesidad de crear nuevas variables.
- **Operaciones en lote:** Las listas permiten realizar operaciones en grupo, como ordenar y filtrar, de manera más eficiente que aplicar operaciones individuales a cada variable.

Arrays en Python

En Python, utilizaremos listas simples para representar arrays, enfocándonos en su uso algorítmico. Las listas almacenan colecciones de objetos de manera ordenada y accesible, siendo útiles para datos relacionados.



- Las listas son estructuras de datos que almacenan elementos en un orden secuencial.
- Los elementos dentro de una lista pueden ser de diferentes tipos (enteros, cadenas, booleanos, etc.)



Características de listas :

- **Mutable:** los elementos de una lista pueden cambiar luego de su creación.
- **Indexable:** los elementos de una lista pueden accederse mediante índices (posiciones).
- **Ordenada:** Los elementos de una lista mantienen un orden específico (no significa que los elementos están ordenados, por ejemplo de menor a mayor)

Problema: gestión de calificaciones de estudiantes

Imaginá que necesitas gestionar las calificaciones de 100 estudiantes en una clase. Usar variables individuales para cada calificación sería impráctico y engorroso. Por ejemplo, si intentas hacerlo con variables individuales, tendrías que definir 100 variables, una para cada estudiante, y luego realizar operaciones en cada una de ellas.

Problema: gestión de calificaciones de estudiantes

Aquí, gestionar y operar con tantas variables individuales es extremadamente complicado y propenso a errores.

```
#Ejemplo impráctico con variables individuales
calificacion1 = 85
calificacion2 = 90
calificacion3 = 78
#...
calificacion100 = 88

#Imagina que necesitas calcular el promedio
promedio = (calificacion1 + calificacion2 + calificacion3 + ... + calificacion100) / 100
```

Solución con listas

Las listas permiten almacenar todas las calificaciones en una sola estructura, lo que facilita la gestión y manipulación de los datos.

```
# Lista de 100 calificaciones (puedes reemplazar estos valores con los tuyos)
calificaciones = [85, 90, 78, 92, ..., 86]

# Inicializar la variable suma en 0
suma = 0

# Ciclo for para acceder a las calificaciones por su índice y sumarlas
for i in range(len(calificaciones)):
    suma += calificaciones[i]

# Calcular el promedio
promedio = suma / len(calificaciones)
```

Explicación

- **Organización:** Con una lista, podés almacenar todas las calificaciones en un solo lugar, en lugar de tener que definir 100 variables.
- **Acceso y Manipulación:** Podés acceder a cualquier calificación usando su índice y realizar operaciones en todas las calificaciones de manera eficiente.
- **Escalabilidad:** Si agregás más estudiantes, solo necesitas añadir elementos a la lista en lugar de crear nuevas variables.

Declaración de una lista vacía en Python

- **Declaración Básica:** Crea una lista vacía usando corchetes

```
lista_vacia = []
```

- **Uso del Constructor:** Alternativamente, usa el constructor 'list()' para crear una lista vacía

```
lista_vacia = list()
```

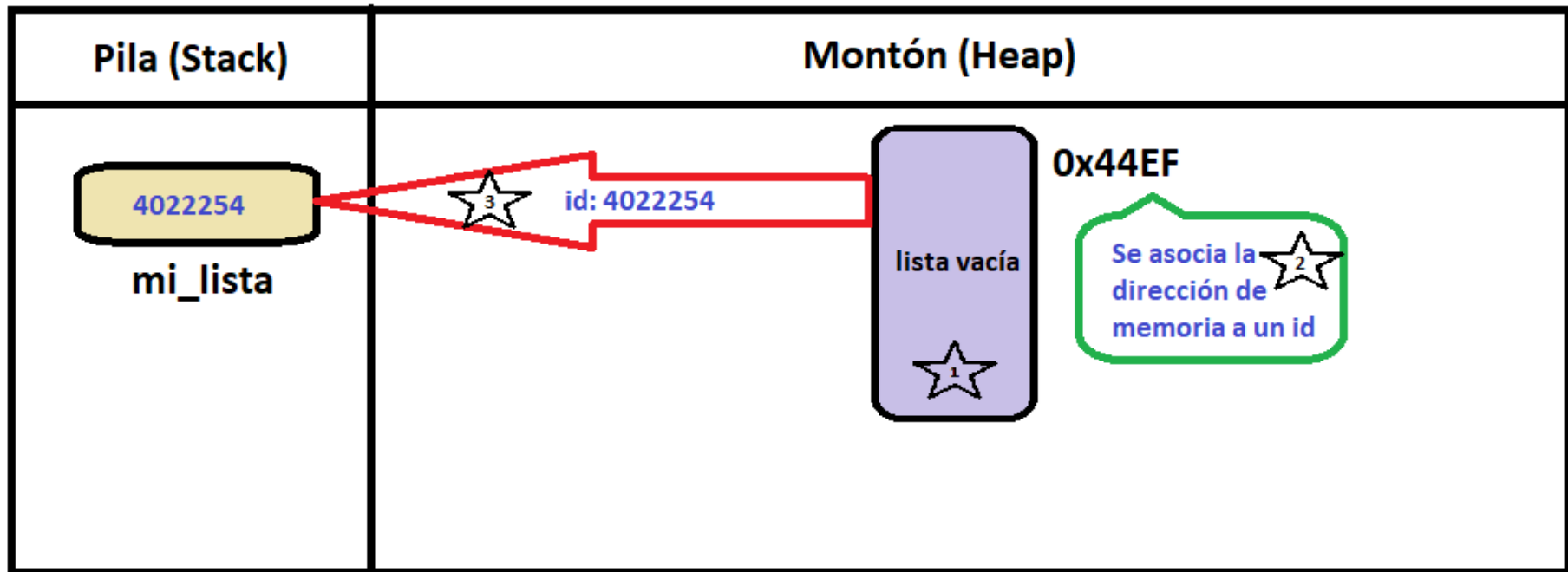
Comportamiento en la memoria - Stack y Heap

Estas instrucciones crean una lista vacía y asigna esa lista a la variable `mi_lista`.

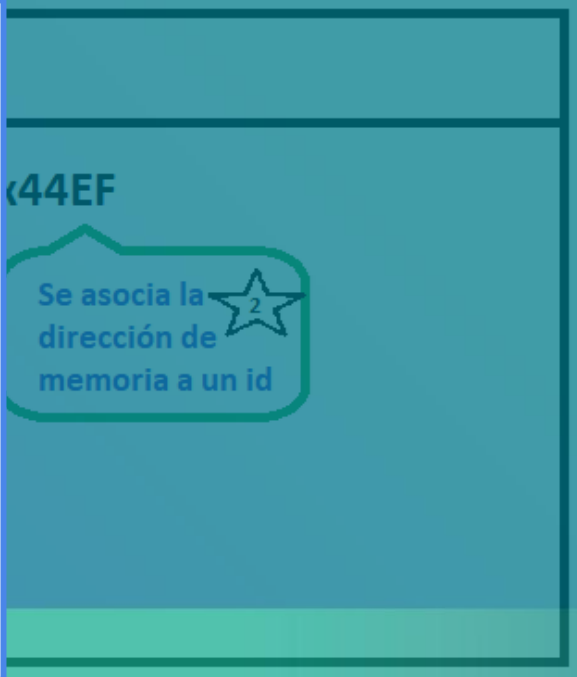
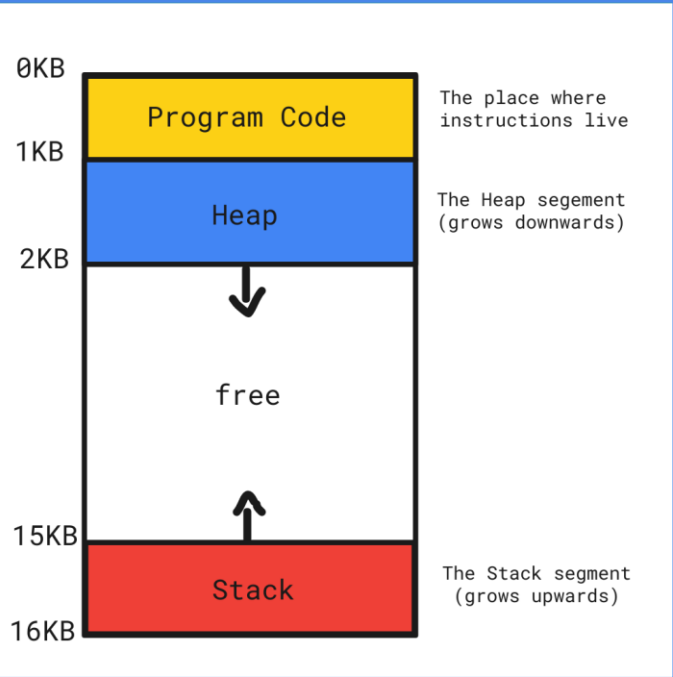
¿Qué sucede en memoria?

1. Se reserva un espacio en memoria para almacenar la lista vacía. Este espacio contendrá la información necesaria para administrar la lista, como su tamaño, capacidad (espacio asignado en memoria) y una referencia a los elementos de la lista.
2. Se asigna un identificador único a esta lista en memoria (su id)
3. Se asocia la variable con este espacio de memoria que contiene la lista vacía en memoria.

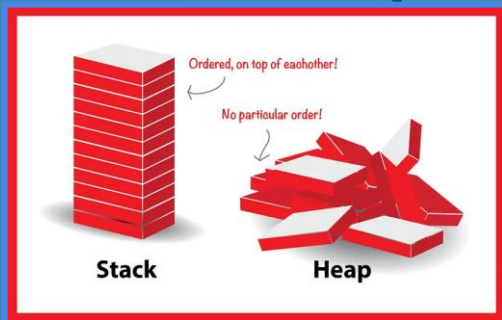
Memoria RAM



Memoria RAM



Pila (Stack)



	Stack (Pila)	Heap (Montón)
Asignación	<i>Estática: es una región de memoria de tamaño fijo y predefinido (por lo general muy pequeño respecto de los otros segmentos) que se asigna en tiempo de ejecución.</i>	<i>Dinámica: es una región de memoria más grande que la pila y dinámica que se asigna cuando se solicita durante la ejecución del programa. Es decir, las variables pueden cambiar su tamaño. Por ejemplo una lista.</i>
Almacenamiento	<i>Variables locales: se utiliza para almacenar valores locales y retornos de una función.</i>	<i>Almacenamiento de objetos, listas, diccionarios</i>
Gestión	<i>Es gestionada de manera automática en tiempo de ejecución</i>	<i>Es gestionada de manera manual o automática (dependiendo del lenguaje). Manual: lo hace el programador. El mismo es encargado de reservar y liberar la memoria para una variable determinada. Por ejemplo en C. Automática: lo gestiona el lenguaje. El mismo reserva espacio en memoria dinámica y al momento de liberar la misma se apoya en un software denominado Recolector de Basura (Garbage Collector). Por ejemplo C#, Python, etc.</i>
Acceso	<i>Funcionamiento LIFO (último entrado, primero salido)</i>	<i>El acceso no es secuencial. No sigue un orden específico.</i>
	<i>Las operaciones de heap pueden ser más lentas que en el stack debido a la necesidad de asignar y liberar memoria de manera dinámica.</i>	

Inicialización de una lista

Definición de una lista utilizando corchetes:

```
nombres = ['Rosita', 'Manuel', 'Luciana']  
nums = [92, 38, 42, 65, 11]
```

Inicializar a 0 una lista de 5 elementos:

```
#Inicializar una lista de 5 elementos con valores 0  
mi_lista = [0] * 5  
  
#Ciclo for para asignar a cada posición de la lista el valor de su índice  
for i in range(len(mi_lista)):  
    mi_lista[i] = i
```

Manipulación de elementos usando índices

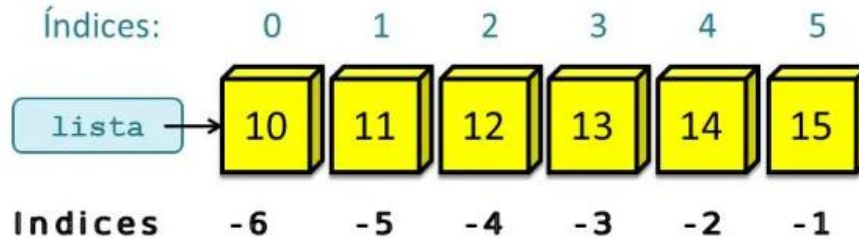
- Las listas se indexan de forma automática, donde el primer elemento está en la posición 0, el segundo en la posición 1, y así sucesivamente

```
#Creación de la lista
```

```
lista = [10, 11, 12, 13, 14, 15]
```

Manipulación de elementos usando índices

```
lista = [10, 11, 12, 13, 14, 15]
```



Índice del primer elemento: 0

Índice del último elemento: 5

Cantidad de elementos: 6

Manipulación de elementos usando índices

```
1  mi_lista = [10, "hola", True, 3.5]
2
3  # Acceder al primer elemento
4  print(mi_lista[0]) # Output: 10
5
6  # Acceder al segundo elemento
7  print(mi_lista[1]) # Output: hola
8
9  # Acceso con índice negativo (último elemento)
10 print(mi_lista[-1]) # Output: 3.5
11
```

Modificación de elementos por índice

Modificar un elemento a través del índice :

```
1  mi_lista = [10, "hola", True, 3.5]
2
3  mi_lista[1] = "adiós"
4  print(mi_lista) # Output: [10, "adiós", True, 3.5]
5
```

Función len()

- Las listas en Python son dinámicas, lo que significa que puedes agregar o eliminar elementos durante la ejecución.
- Para verificar la longitud (cantidad de elementos) actual de una lista, se utiliza la función len().
- La función len() toma el nombre de la lista como argumento y devuelve un número entero representando la cantidad de elementos dentro de la lista.

Función len()

```
1  # Definir una lista
2  lista = [10, 11, 12, 13, 14, 15]
3
4  # Obtener la longitud de la lista
5  longitud = len(lista)
6
7  # Imprimir la longitud de la lista
8  print("La longitud de la lista es:", longitud)
```

Mostrar una lista



```
mi_lista = [4, 9, 3, 7, 1]  
  
print(mi_lista)
```



```
mi_lista = [4, 9, 3, 7, 1]  
  
for i in range(len(mi_lista)):  
    print(mi_lista[i])
```

Usando el bucle for para procesar listas

- El bucle for en Python es una herramienta efectiva para recorrer y procesar listas.
- Para calcular la suma de todos los elementos de una lista, se puede usar una variable acumuladora (en este caso total) y un bucle que recorra los elementos de la lista.

Nota: Es una buena práctica evitar el uso de nombres como sum para variables, ya que esto puede entrar en conflicto con funciones integradas como sum().

Usando el bucle for para procesar listas

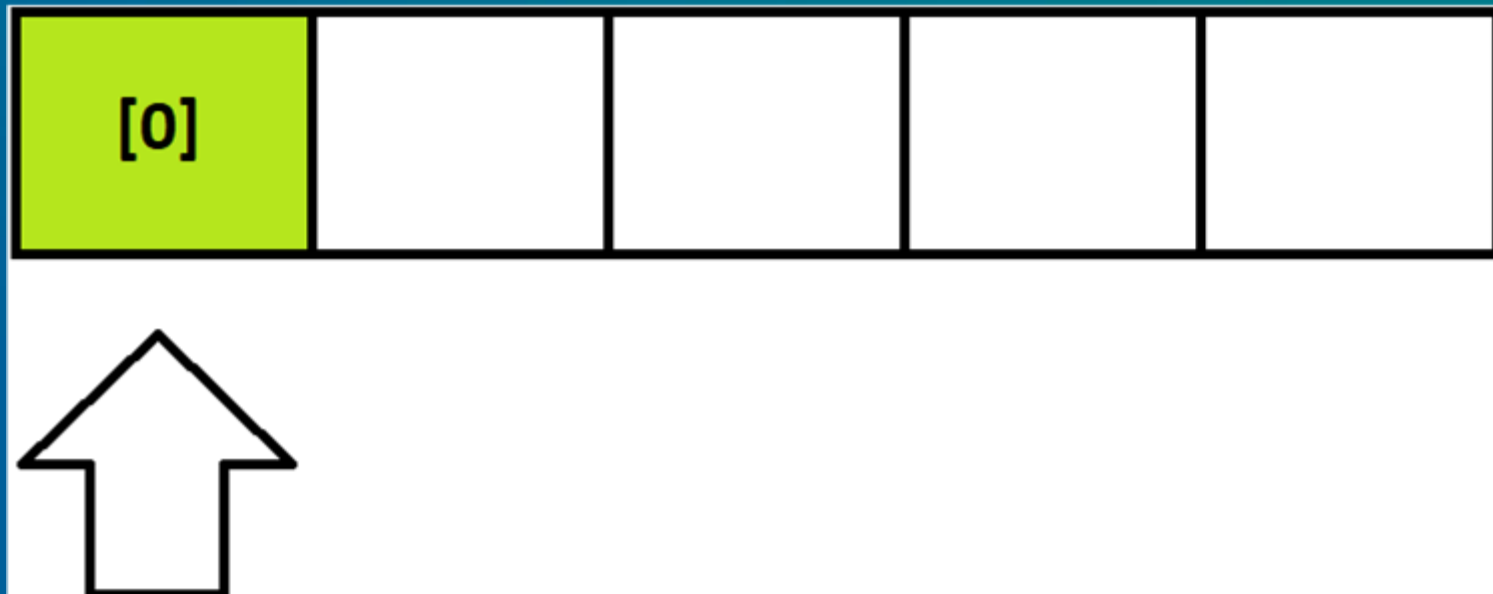
```
#Lista con 5 valores enteros
my_list = [1, 2, 3, 4, 5]

#Variable acumuladora para la suma
total = 0

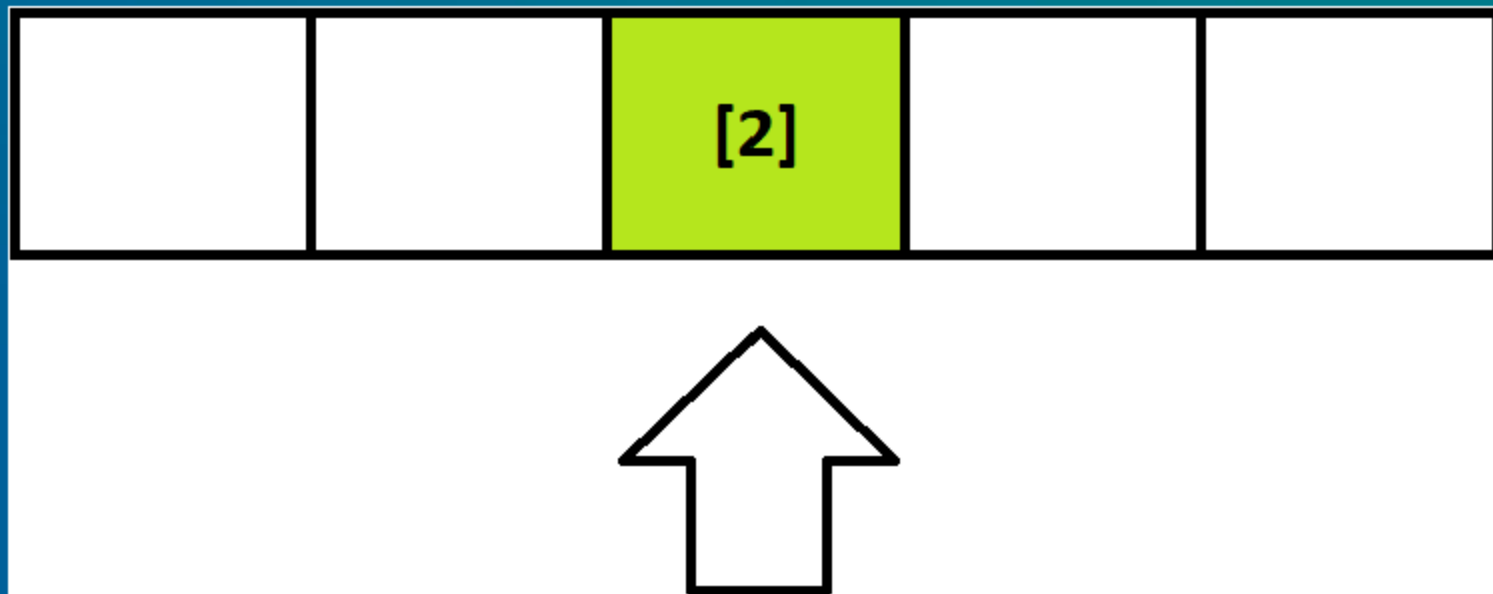
#Usar un bucle for para recorrer la lista y sumar sus elementos
for i in range(len(my_list)):
    total += my_list[i]

#Imprimir el resultado de la suma
print("La suma total es:", total )
```

Cargar una lista de forma secuencial



Cargar una lista con acceso aleatorio



Cargar una lista con acceso aleatorio

Definir manualmente la posición y valor para cargar en una lista.

- El usuario puede definir tanto el valor que desea agregar a la lista como la posición en la que desea insertarlo.
- Esto proporciona al usuario control total sobre la estructura y contenido de la lista.

Operaciones en listas



Características clave de las listas

Las listas se comportan de manera diferente a las variables ordinarias (escalares).

Ejemplo sorprendente:

- Crea una lista de un elemento llamada `list_1`.
- Asigna `list_1` a una nueva lista llamada `list_2`.
- Cambia el único elemento de `list_1`.
- Imprime `list_2`.

Resultado inesperado: `list_2` mostrará `[2]`, no `[1]`.

Razón: Las listas y otras entidades complejas se almacenan de manera diferente que las variables ordinarias.

Características clave de las listas

```
#Crea una lista de un elemento llamada list_1
list_1 = [1]

#Asignar list_1 a una nueva lista llamada list_2
list_2 = list_1

#Cambia el único elemento de list_1
list_1[0] = 2

#Imprime la list_2
print(list_2) #Esto mostrará [2], no [1]
```

Entender la asignación en listas

Cómo funciona la asignación:

Variables ordinarias:

- El nombre de la variable es el nombre de su contenido.

Listas:

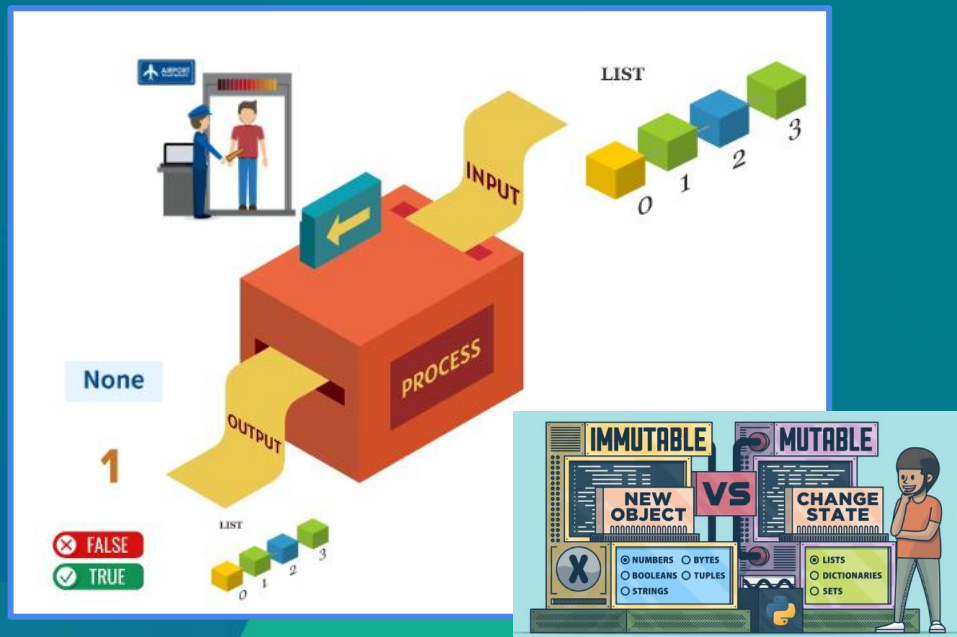
- El nombre de una lista es el nombre de una ubicación de memoria (su id) donde se almacena la lista.

Entender la asignación en listas

- **Ejemplo clave:**
 - `lista_2 = lista_1` copia el nombre del arreglo, no su contenido.
 - Ambos nombres (`lista_1` y `lista_2`) identifican la misma ubicación en la memoria.
- **Implicación: modificar uno afecta al otro.**

Listas y funciones

- Las listas son objetos parametrizables, esto significa que es posible pasarlas como parámetros de una función.
- Siempre es recomendable realizar un control de tipo sobre el parámetro, es decir verificar si el type del parámetro es del tipo list.
- Según su procesamiento, estas funciones podrán retornar todos los tipos conocidos, incluyendo otras listas.
- No olvidar el carácter mutable de las listas.



Algoritmos de búsqueda

Según la naturaleza de la búsqueda, es posible implementar distintos algoritmos sobre las listas.

